



Especificación de Requerimientos – Colitas felices

Iteración: 3

Alumnos: Julieta Melina Flores Morán (321088642) - José Antonio Zarco Romero (321102502)- Luis Ángel Casique Abraham (321266536) - Yolanda Ximena Sánchez Cervantes (319263886)

Profesora: Karla Socorro García Alcántara

Versión del Documento: 3.0

Fecha: 8 de mayo de 2026

1. Introducción

1.1 Propósito del documento

Este documento describe la evolución de los requerimientos funcionales y no funcionales del sistema Colitas Felices en el contexto de la Iteración 3 del proyecto. Su propósito es reflejar el estado actual del sistema implementado, funcionando como referencia técnica y funcional tanto para el equipo de desarrollo como para stakeholders.

Esta versión documenta el sistema completo que integra back-end e interfaz de usuario con funcionalidades extendidas más allá del manejo de usuarios. La Iteración 3 implementa las funcionalidades básicas de un cuidador cómo publicar, consultar, editar, eliminar fichas de animales y marcar como adoptado; y las de un adoptante como explorar animales disponibles y manifestar interés, logrando que esta acción notifique al cuidador por correo electrónico y que ambas partes continúen la comunicación de forma autónoma.

Asimismo, este documento sirve como base de conocimiento para futuras iteraciones, permitiendo mantener coherencia entre el diseño del sistema y su implementación real.

1.2 Alcance del sistema

El sistema Colitas Felices es una aplicación web orientada a facilitar la adopción responsable de perros y gatos en el territorio mexicano, sirviendo como punto de contacto entre entidades cuidadoras de animales y personas interesadas en adoptar. Cuando un usuario hace clic en "Me interesa", el sistema envía automáticamente una notificación por correo al cuidador con

los datos de contacto del interesado, permitiendo que ambas partes continúen el proceso de adopción de manera autónoma.

En la Iteración 3, el sistema evoluciona incorporando los requerimientos RF6–RF8 y RF10–RF13. Las funcionalidades actualmente implementadas corresponden a:

- Gestión de usuarios: registro, inicio de sesión, cierre de sesión, consulta y actualización de perfil (RF1–RF5)
- Gestión de animales: publicación, consulta, edición, eliminación y cambio de estado a adoptado (RF6, RF7, RF8)
- Visualización de animales disponibles mediante lista (RF10)
- Ficha informativa con datos completos del animal (RF11)
- Manifestación de interés con notificación por correo al cuidador y notificación al retirar interés (RF12)
- Apartado de "Mis me gusta" con lista de animales favoritos (RF13)
- Almacenamiento de imágenes mediante Cloudinary (perfil y animales)

Estas funcionalidades permiten tener un sistema básico de adopciones funcional

Fuera del alcance en la Iteración 3 del sistema

Las siguientes funcionalidades, aunque contempladas para el producto final, no forman parte del sistema actualmente implementado:

- Visualización de animales mediante mapa interactivo
- Funcionalidades administrativas
- Integración con API externa de razas
- Verificación de correo electrónico en registro
- Mecanismo de recuperación de contraseña
- Verificación en dos pasos para inicio de sesión

- Notificación de ingreso exitoso al correo del usuario
- Eliminación de cuenta
- Historial de animales adoptados
- Bloqueo por intentos fallidos de inicio de sesión
- Logs de actividad
- Guía informativa
- Filtros y ordenamiento avanzados en lista de animales

1.3 Definiciones relevantes

1. **Usuario:** Persona registrada que accede al sistema con credenciales (rol: cuidador o interesado).
 - a. **Cuidador:** Usuario que publica y gestiona fichas de animales en adopción.
 - b. **Interesado:** Usuario que busca adoptar un animal.
2. **Usuario nuevo:** Persona que no cuenta con una cuenta registrada y desea crear un perfil dentro del sistema.
3. **Información personal:** Datos asociados a un usuario que permiten su identificación dentro del sistema, tales como nombre, correo electrónico y otros atributos definidos en el modelo de datos.
4. **Rol de usuario:** Atributo asociado a cada usuario que indica su tipo dentro del sistema (por ejemplo, cuidador o interesado). En la Iteración 1, este atributo es almacenado, pero no define comportamientos diferenciados dentro del sistema.
5. **Autenticación:** Proceso mediante el cual el sistema verifica la identidad de un usuario a partir de sus credenciales.
6. **Usuario autenticado:** Usuario que ha iniciado sesión correctamente y cuenta con acceso a funcionalidades protegidas del sistema.
7. **Manifestación de interés:** Acción mediante la cual un adoptante expresa interés en un animal, generando una notificación automática por correo al cuidador.

2. Actores del sistema

Actor: Usuario registrado. **Tipo:** Interno. **Descripción:** Persona que crea una cuenta dentro del sistema y accede a la plataforma utilizando sus credenciales para consultar información sobre animales disponibles para adopción y utilizar las funcionalidades que le ofrece el sistema. **Responsabilidades dentro del sistema:**

- Registrarse en la plataforma proporcionando la información requerida.
- Iniciar sesión mediante sus credenciales.
- Gestionar la información de su perfil personal.
- Cerrar sesión.

Actor: Usuario cuidador (publicador de animales). **Tipo:** Interno. **Descripción:** Usuario registrado responsable de publicar y administrar las fichas de animales que se encuentran bajo su cuidado y que están disponibles para adopción dentro de la plataforma. **Responsabilidades dentro del sistema:**

- Registrar animales disponibles para adopción.
- Ingresar la información correspondiente de cada animal (nombre, descripción, fotografías, raza, edad, estado de vacunación, esterilización, padecimientos y género).
- Editar la información de los animales registrados.
- Eliminar publicaciones de animales cuando ya no estén disponibles.
- Marcar animales como adoptados.
- Consultar la lista de animales que ha registrado, incluyendo fecha de publicación y número de interesados.

Actor: Usuario interesado en adopción (adoptante). **Tipo:** Interno. **Descripción:** Usuario registrado que utiliza la plataforma para explorar animales disponibles y expresar interés en adoptar alguno de ellos. **Responsabilidades dentro del sistema:**

- Explorar animales disponibles mediante la vista de lista.

- Consultar la ficha informativa de los animales disponibles.
- Manifestar su interés en adoptar un animal mediante el botón "Me interesa".
- Retirar su manifestación de interés en cualquier momento.
- Consultar los animales a los cuales ha manifestado interés (mis favoritos).

Actor: Sistema de correo electrónico. Tipo: Externo. Descripción: Servicio externo encargado de enviar notificaciones automáticas por correo electrónico cuando un usuario manifiesta o retira interés en un animal. Responsabilidades dentro del sistema:

- Enviar notificación al cuidador cuando un adoptante manifiesta interés en uno de sus animales, incluyendo los datos de contacto del interesado.
- Enviar notificación al cuidador cuando un adoptante retira su interés.
- Incluir al adoptante en copia en ambas notificaciones.

3. Requerimientos Funcionales

Los siguientes requerimientos funcionales corresponden a las funcionalidades implementadas en la Iteración 3 del sistema. En esta iteración se cubre la gestión completa de usuarios, la gestión de animales en adopción por parte de cuidadores, la visualización del catálogo de animales disponibles y la manifestación de interés por parte de adoptantes con notificación automática por correo electrónico.

Requerimiento		Descripción	Criterios de aceptación
Gestión de usuarios			
RF1	Registro de usuarios	El sistema deberá permitir a nuevos usuarios registrarse en la plataforma mediante el ingreso de su información personal, con el fin de crear una cuenta única.	El sistema deberá solicitar la información necesaria para el registro del usuario. El sistema deberá validar que la información obligatoria esté completa. El registro requiere: nombre, nombre de usuario, correo electrónico, código postal, CURP, rol (cuidador o adoptante) y contraseña. El sistema deberá validar que el correo electrónico y el nombre de usuario no estén previamente registrados. El sistema deberá registrar al usuario en la plataforma si la información es válida. Las contraseñas deberán

			cumplir con requisitos mínimos de longitud y complejidad de caracteres.
RF2	Inicio de sesión	El sistema deberá permitir a los usuarios registrados autenticarse para acceder a la plataforma.	El sistema deberá solicitar credenciales de acceso al usuario. El sistema deberá validar que las credenciales correspondan a un usuario registrado. El sistema deberá permitir el acceso únicamente cuando las credenciales sean correctas. El sistema deberá rechazar el acceso cuando las credenciales sean inválidas. El ingreso se realizará con correo electrónico y contraseña.
RF3	Cierre de sesión	El sistema deberá permitir a los usuarios cerrar su sesión activa en la plataforma.	El sistema deberá finalizar la sesión del usuario autenticado. El sistema deberá impedir el acceso a funcionalidades protegidas después del cierre de sesión.
RF4	Obtener usuario autenticado	El sistema deberá permitir al usuario autenticado consultar su información personal.	El sistema deberá permitir el acceso únicamente a usuarios autenticados. El sistema deberá mostrar la información correspondiente al usuario autenticado. El sistema no deberá permitir el acceso a información de otros usuarios.
RF5	Actualización de datos de usuario autenticado	El sistema deberá permitir al usuario autenticado actualizar su información personal.	El sistema deberá permitir únicamente a usuarios autenticados realizar la actualización. El sistema deberá validar la información enviada y guardarla. Debe permitirles cambiar su información personal: nombre(s), apellido paterno, apellido materno, código postal, correo y foto de perfil.
Gestión de animales			
RF6	Publicación de animales en adopción	El sistema deberá permitir a los usuarios con rol CUIDADOR crear un perfil de animal disponible para adopción.	Registrar publicaciones de perfil de animal en adopción con los siguientes datos mínimos: nombre del animal, descripción general, fotografía(s) (al menos una), tipo de animal (perro o gato), raza o raza similar, edad aproximada, vacunación, esterilización, padecimientos y género. El sistema deberá asociar esta información con el perfil del cuidador que lo publica y su código postal. Todo animal registrado es visible para que lo descubran los adoptantes.
RF7	Control de estado de publicación	El sistema deberá permitir a los cuidadores marcar los animales que	Solo el usuario que registró el animal puede cambiar su estado a "Adoptado" desde la gestión de

		publicaron como adoptados, retirándolos de la lista pública de adopción.	sus publicaciones. Los animales marcados como adoptados dejan de aparecer en la lista de animales disponibles. Los animales adoptados permanecen visibles en la lista personal del cuidador ("Mis mascotas") con indicador visual de su estado, pudiendo filtrarse por estatus. El cuidador puede revertir el estado a DISPONIBLE.
RF8	Lista de animales registrados por usuario	El sistema deberá permitir a los cuidadores consultar y administrar los animales que han registrado.	Solo debe ser visible para el cuidador propietario. Debe permitirle ver los animales que registró. Debe permitir editar y eliminar los animales registrados. Puede visualizar fecha de publicación y número de interesados por cada animal.
Consulta de animales			
RF10	Lista de animales disponibles	El sistema deberá ofrecer una vista de catálogo para que los usuarios naveguen por la totalidad de los animales registrados con estatus DISPONIBLE.	El sistema muestra una lista con todos los animales registrados disponibles para adopción. El usuario puede seleccionar un animal de la lista para acceder a su ficha informativa completa.
RF11	Ficha informativa	El sistema deberá mostrar una ficha informativa detallada para cada animal registrado en la plataforma.	Cada ficha debe mostrar de forma clara: nombre, descripción, fotografía(s) con galería navegable, tipo de animal, raza, código postal, edad, género, estado de vacunación, esterilización y padecimientos. Para adoptantes: debe incluir un botón visible denominado "Me interesa" para iniciar el proceso de contacto. Para el cuidador propietario: debe incluir controles de edición, eliminación y cambio de estado.
RF12	Manifestar interés en un animal	El sistema maneja de forma automática el envío de correo de un usuario adoptante interesado en adoptar hacia el cuidador que ha publicado un animal.	Al hacer clic en "Me interesa", el sistema deberá enviar automáticamente una notificación al correo del cuidador con los datos de contacto del adoptante, incluyendo al adoptante en copia. El usuario interesado deberá poder retirar su manifestación de interés en cualquier momento y esto también debe ser notificado al cuidador por correo. Si el envío de correo falla al manifestar interés, el sistema hace rollback y no registra el interés. Si el envío falla al retirar interés, el retiro se completa igualmente.

RF13	Apartado de mis favoritos	El sistema debe mostrar al adoptante los animales en los que ha manifestado interés para seguimiento rápido.	El sistema mantiene una lista de animales favoritos del adoptante. El usuario puede retirar su interés desde esta vista, eliminando el animal de la lista.
------	---------------------------	--	--

4. Casos de Uso

CU1 – Registro de usuario (RF1)

Actores:

- Usuario nuevo

Descripción:

Este caso de uso describe el proceso mediante el cual un usuario crea una cuenta en la plataforma enviando su información al sistema.

Precondiciones:

- El usuario no cuenta con una cuenta registrada.
- El usuario puede realizar solicitudes al sistema.

Flujo Principal:

1. El usuario accede a la vista de registro y envía una solicitud de registro con su información.
2. El sistema recibe la solicitud.
3. El sistema verifica que la información esté completa y cumpla con el formato esperado.
4. El sistema verifica que el correo, CURP y nombre de usuario no estén registrados.
5. El sistema cifra la contraseña.
6. El sistema registra al usuario en la base de datos.
7. El sistema confirma el registro exitoso y redirige al usuario a la vista de inicio de sesión.

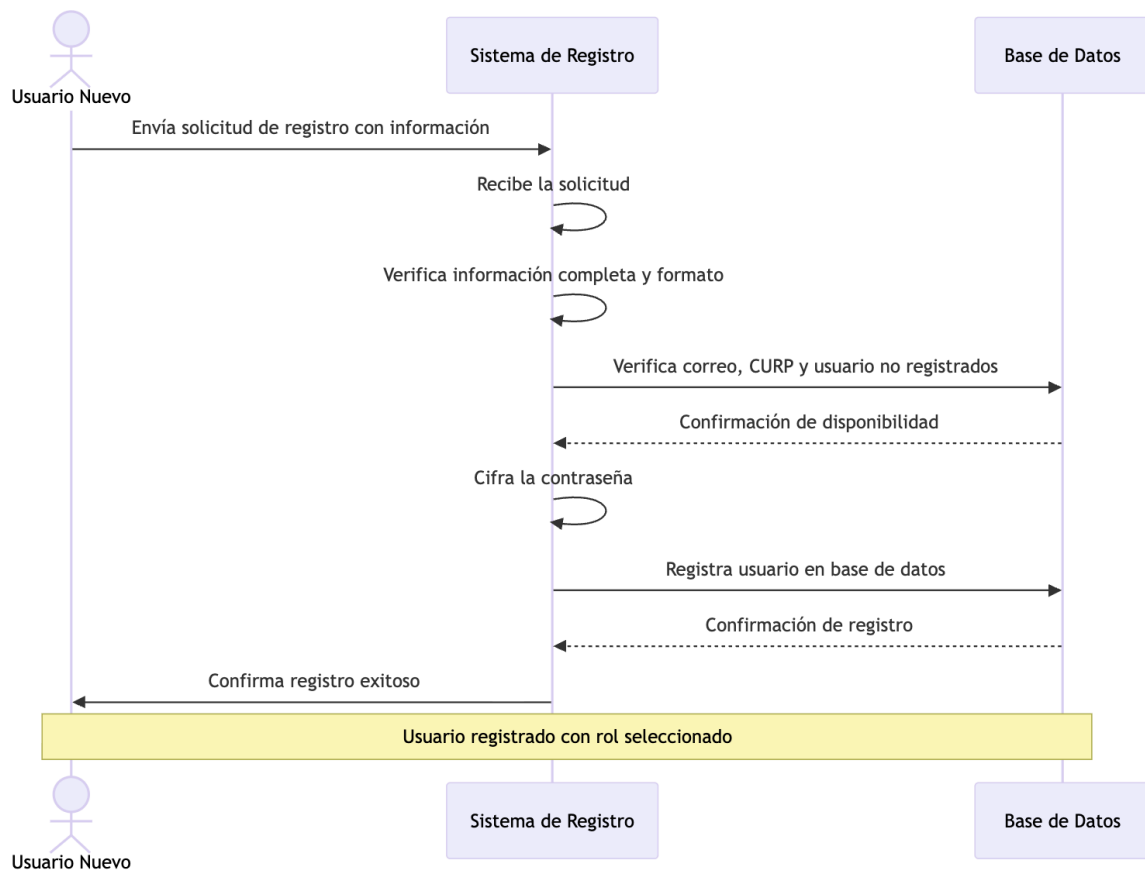
Flujos Alternativos:

- A1 – Información obligatoria faltante: Si el usuario omite uno o más campos obligatorios, el sistema muestra un mensaje de error indicando los campos pendientes. El sistema no registra al usuario.
- A2 – Correo electrónico o nombre de usuario ya registrado: Si el correo ingresado o el nombre de usuario ya existe en el sistema, el sistema muestra un mensaje indicando que el correo o nombre de usuario ya está en uso. El usuario deberá ingresar un correo, nombre de usuario diferente. El sistema no registra al usuario.
- A3 – Formato de entrada inválido: Si los datos ingresados no tiene un formato válido, el sistema muestra un mensaje de error. El sistema no registra al usuario.
- A4 – Contraseña no cumple requisitos: Si la contraseña no cumple con los requisitos mínimos de seguridad, el sistema muestra un mensaje indicando los criterios que deben cumplirse. El sistema no registra al usuario.

Postcondiciones:

- La cuenta del usuario queda registrada en el sistema con el rol seleccionado.

Diagrama:



CU2 – Inicio de sesión (RF2)

Actores:

- Usuario registrado

Descripción: Este caso de uso describe el proceso mediante el cual un usuario se autentica en el sistema.

Precondiciones:

- El usuario cuenta con una cuenta registrada y activa

Flujo Principal:

1. El usuario accede a la vista de login y envía una solicitud de autenticación con sus credenciales.

2. El sistema recibe la solicitud.
3. El sistema valida las credenciales.
4. El sistema confirma la autenticación y redirige al usuario a la vista principal (home).

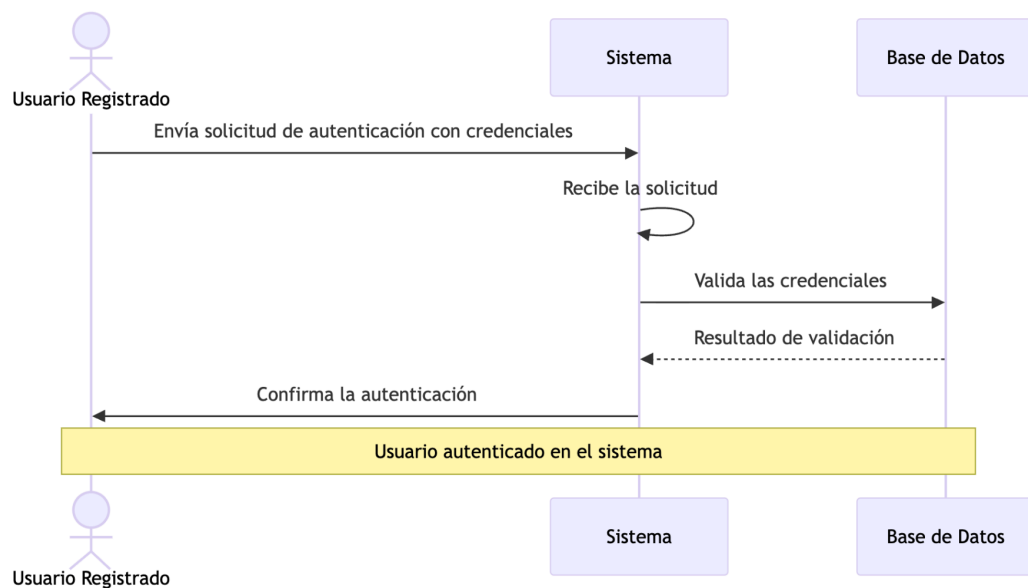
Flujos Alternativos:

- A1 – Credenciales incorrectas: Si el correo o la contraseña ingresados no coinciden con ningún registro, el sistema muestra un mensaje de error genérico sin especificar cuál campo es incorrecto. El sistema rechaza la solicitud de autenticación.

Postcondiciones:

- El usuario queda autenticado en el sistema.

Diagrama:



CU3 – Obtener usuario autenticado (RF4)

Actores:

- Usuario autenticado

Descripción: Este caso de uso describe cómo un usuario autenticado solicita su información personal al sistema.

Precondiciones:

- El usuario está autenticado.

Flujo Principal:

1. El usuario accede a la vista de perfil y envía una solicitud para obtener su información.
2. El sistema valida la autenticación del usuario.
3. El sistema recupera la información del usuario.
4. El sistema devuelve la información solicitada y se la muestra al usuario en la vista de perfil.

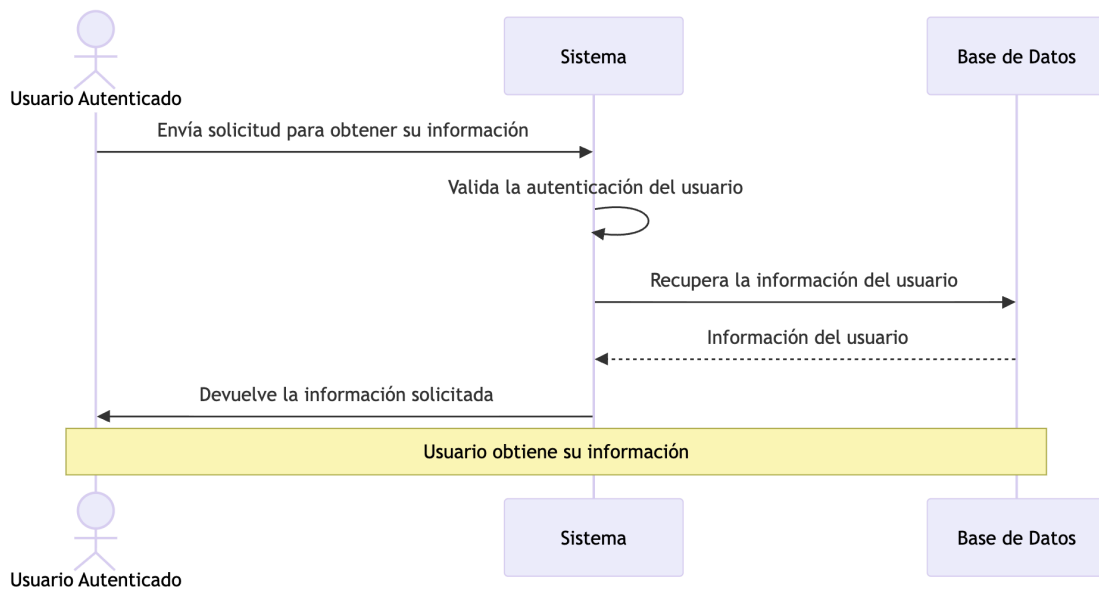
Flujos Alternativos:

- A1 - Usuario no autenticado: Se manda un mensaje de error explicando el problema y la acción de solución (autenticarse). El sistema rechaza la solicitud.

Postcondiciones:

- El usuario obtiene su información.

Diagrama:



CU4 – Actualización de usuario (RF5)

Actores:

- Usuario autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un usuario modifica su información personal dentro del sistema.

Precondiciones:

- El usuario está autenticado.

Flujo Principal:

1. El usuario accede a la vista de perfil y envía una solicitud para editar su perfil.
2. El usuario edita su información personal.
3. El usuario envía la solicitud de actualización.
4. El sistema valida la autenticación del usuario.
5. El sistema valida los datos enviados.
6. El sistema actualiza la información en la base de datos.
7. El sistema confirma la actualización.

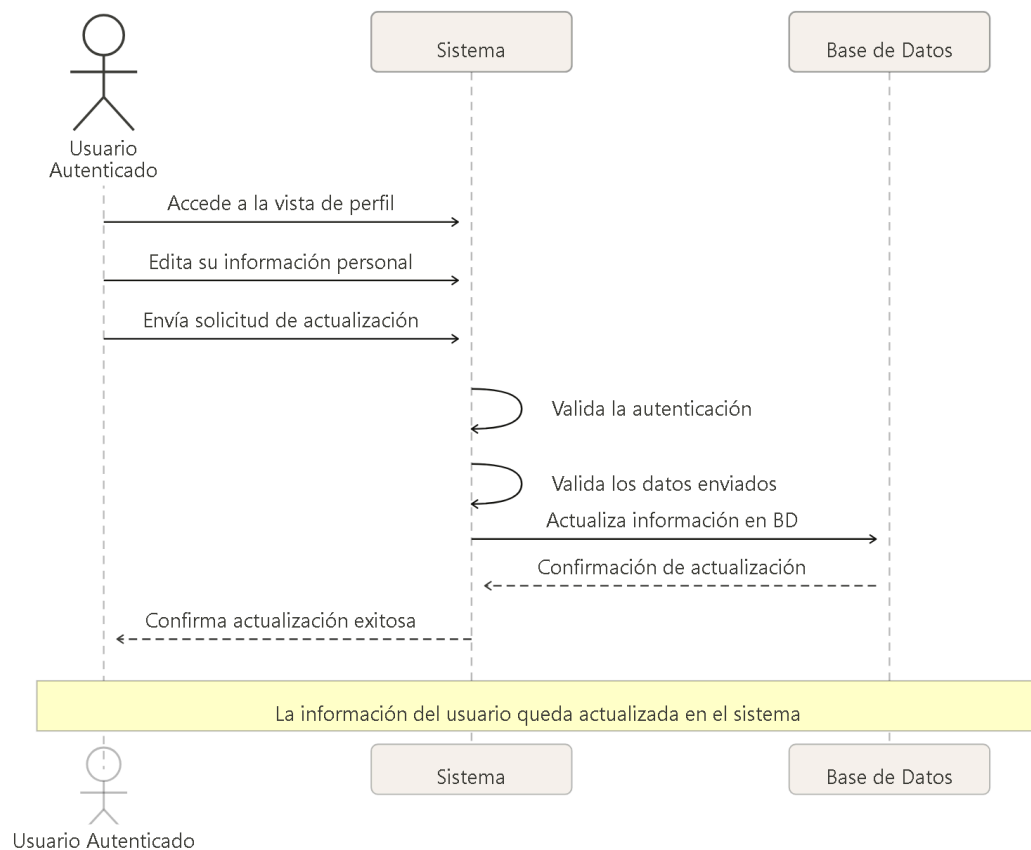
Flujos Alternativos:

- A1 - Usuario no autenticado: Se manda un mensaje de error explicando el problema y la acción de solución (autenticarse). El sistema rechaza la solicitud.
- A2 – Datos inválidos: El sistema muestra un mensaje de error indicando los campos incorrectos.

Postcondiciones:

- La información del usuario queda actualizada en el sistema.

Diagrama:



CU5 – Cierre de sesión (RF3)

Actores:

- Usuario autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un usuario finaliza su sesión en el sistema.

Precondiciones:

- El usuario está autenticado.

Flujo Principal:

1. El usuario selecciona la opción de cerrar sesión desde la barra de navegación y envía una solicitud para cerrar sesión.
2. El sistema recibe la solicitud.
3. El frontend finaliza la sesión del usuario e invalida el token de sessionStorage y redirige al usuario a la vista de login.

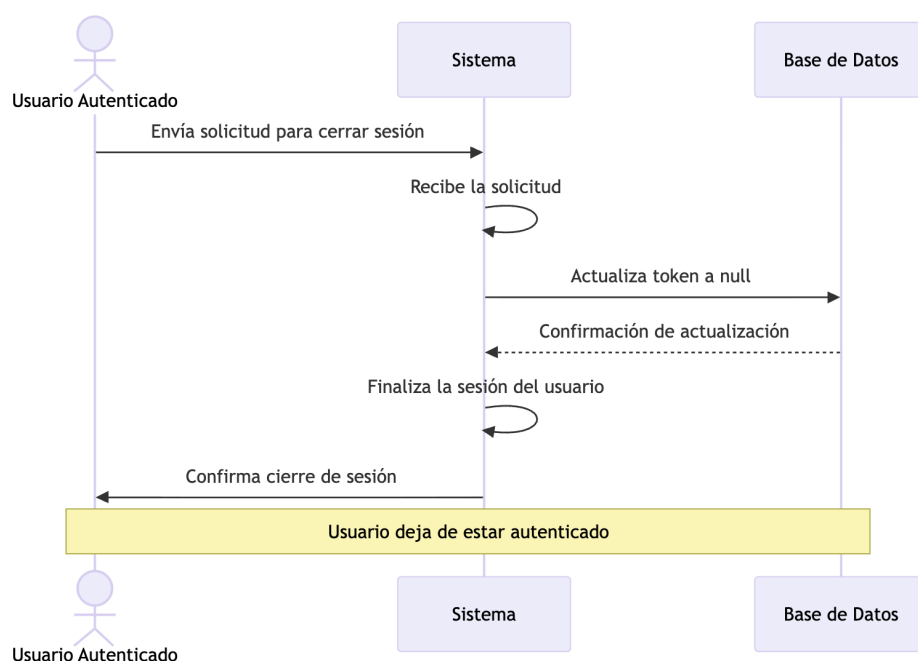
Flujos Alternativos:

- A1 - Usuario no autenticado: Se manda un mensaje de error explicando el problema y la acción de solución (autenticarse). El sistema rechaza la solicitud.

Postcondiciones:

- El usuario deja de estar autenticado.

Diagrama:



CU6 - Publicar animal (RF6)

Actores: Cuidador autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un cuidador registra un nuevo animal en el catálogo del sistema para que esté disponible para adopción.

Precondiciones:

- El usuario está autenticado con rol CUIDADOR.

Flujo Principal:

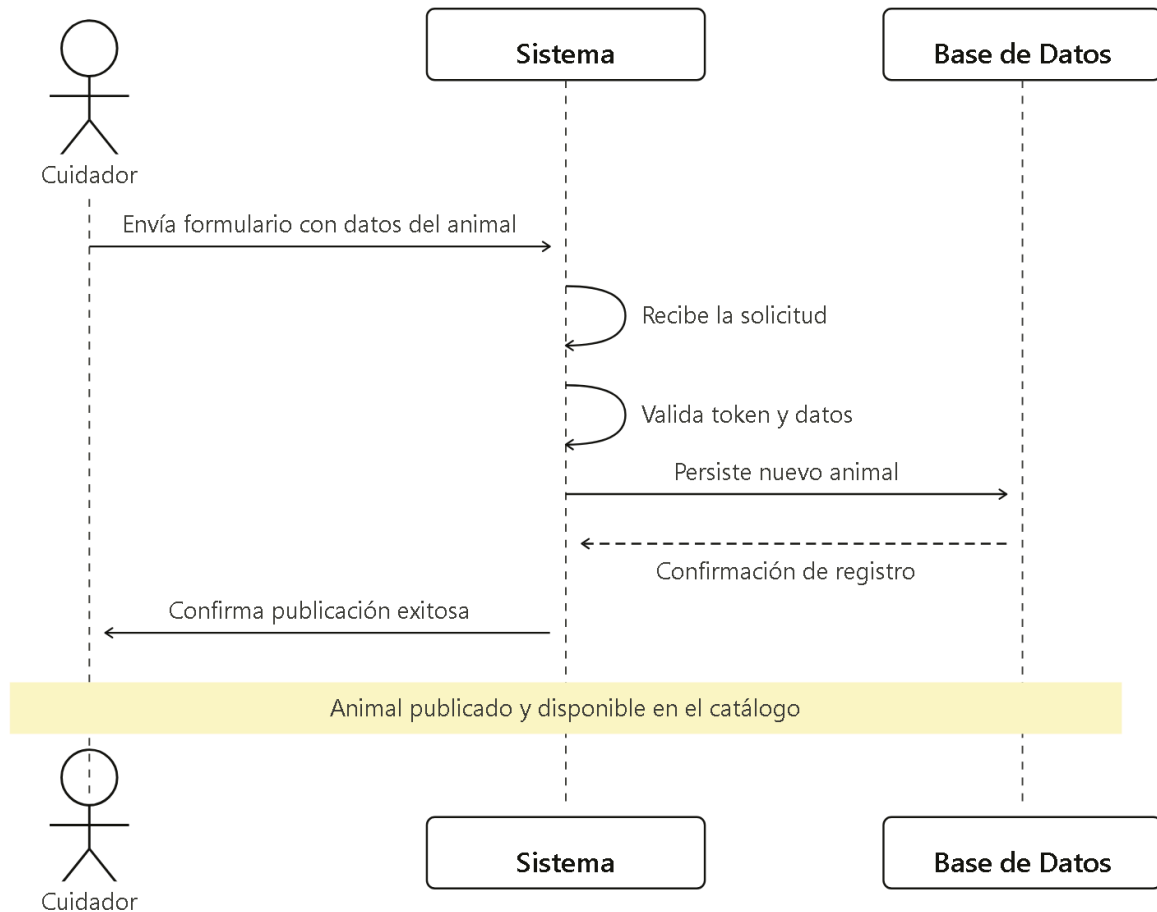
1. El cuidador accede a la vista de publicar animal y completar el formulario con los datos del animal.
2. El sistema recibe la solicitud.
3. El sistema valida el token de autenticación y los datos del formulario.
4. El sistema persiste el nuevo animal en la base de datos con estatus DISPONIBLE.
5. El sistema confirma la publicación exitosa y redirige al cuidador al catálogo.

Flujos Alternativos:

- A1 – Token ausente o inválido: El sistema devuelve un error 401 y rechaza la solicitud.
- A2 – Datos incompletos o inválidos: El sistema devuelve un error 400 indicando los campos incorrectos. El sistema no registra el animal.

Postcondiciones:

- El animal queda registrado en el sistema con estatus DISPONIBLE y es visible en el catálogo.



CU7 - Listar animales disponibles (RF10)

Actores:

- Usuario autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un usuario accede al catálogo general para visualizar todos los animales que se encuentran disponibles para adopción.

Precondiciones:

- El usuario está autenticado.

Flujo Principal:

1. El usuario accede a la vista de catálogo de animales.

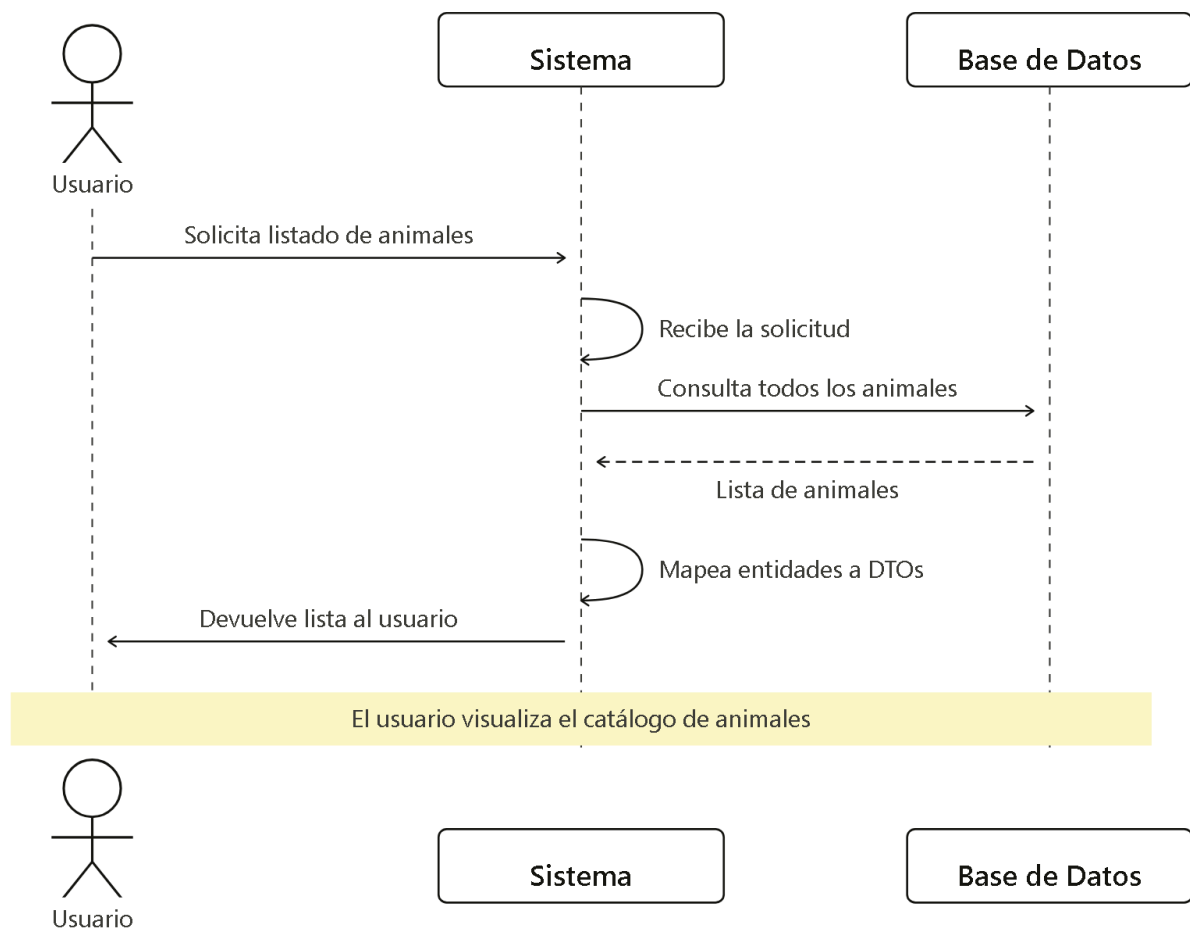
2. El sistema valida la autenticación del usuario.
3. El sistema recupera la lista de animales.
4. El sistema muestra la lista de animales disponibles junto con información resumida de cada publicación.
5. El usuario puede seleccionar un animal para consultar su ficha informativa completa.

Flujos Alternativos:

- A1 – Usuario no autenticado: El sistema muestra un mensaje indicando que debe iniciar sesión. El sistema rechaza la solicitud.
- A2 – No existen animales disponibles: El sistema muestra un mensaje indicando que no hay animales disponibles actualmente.

Postcondiciones:

- El usuario visualiza el catálogo actualizado de animales disponibles para adopción.



CU8 – Editar animal (RF6)

Actores:

- Cuidador autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un cuidador modifica la información de un animal previamente registrado en la plataforma.

Precondiciones:

- El usuario está autenticado con rol CUIDADOR.
- El animal pertenece al usuario autenticado.

Flujo Principal:

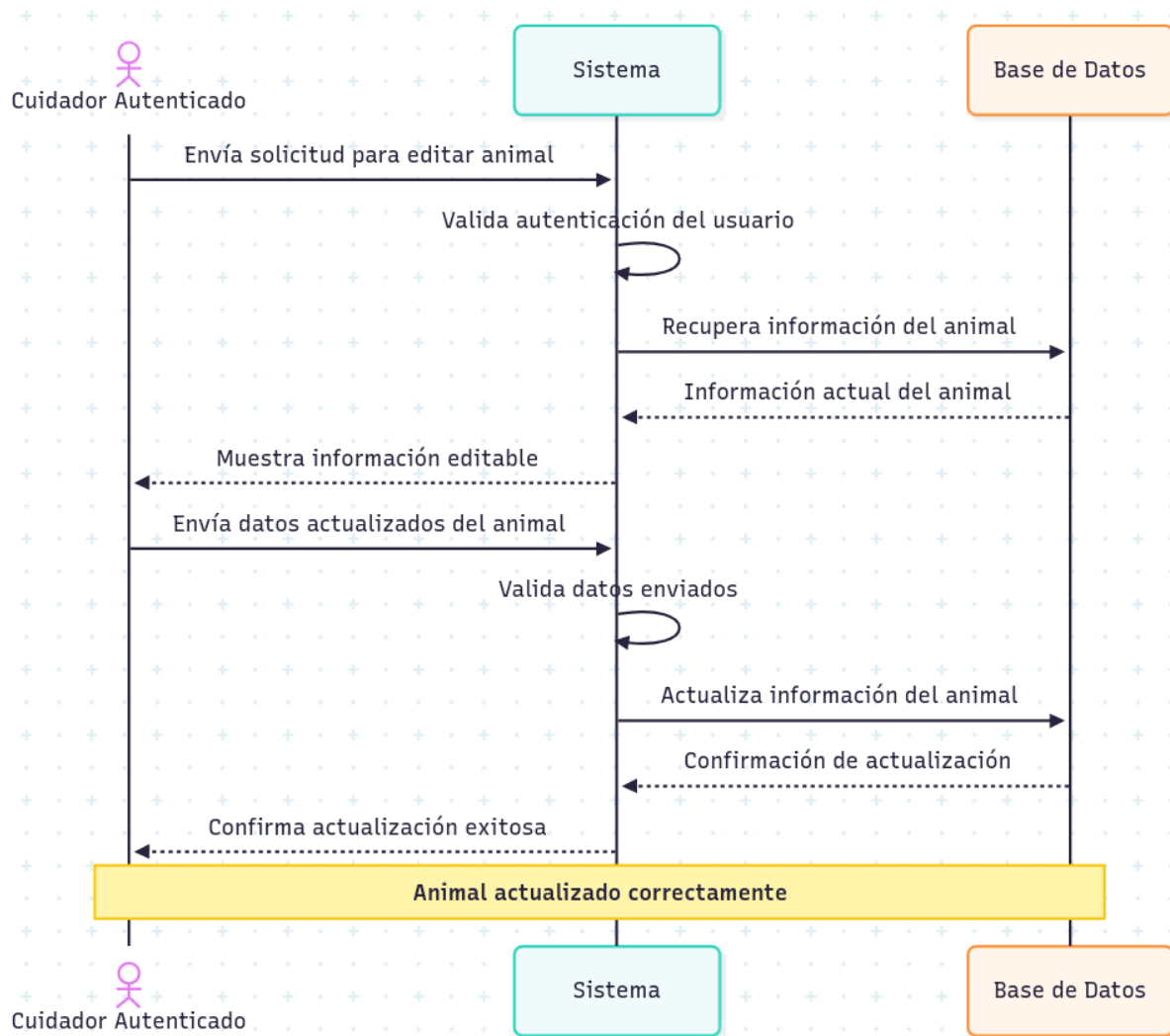
1. El cuidador accede a la vista de administración de sus publicaciones.
2. El cuidador selecciona un animal registrado y solicita editar su información.
3. El sistema recupera la información actual del animal.
4. El cuidador modifica los datos deseados.
5. El sistema valida la autenticación y los datos enviados.
6. El sistema actualiza la información del animal en la base de datos.
7. El sistema confirma la actualización exitosa.

Flujos Alternativos:

- A1 – Usuario no autenticado: El sistema devuelve un error de autenticación y rechaza la solicitud.
- A2 – Animal no pertenece al usuario: El sistema devuelve un error de autorización y no permite la modificación.
- A3 – Datos inválidos o incompletos: El sistema muestra un mensaje indicando los campos incorrectos. El sistema no actualiza la información.

Postcondiciones:

- La información del animal queda actualizada en el sistema.



CU9 – Eliminar animal (RF6)

Actores:

- Cuidador autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un cuidador elimina una publicación de animal registrada en la plataforma.

Precondiciones:

- El usuario está autenticado con rol CUIDADOR.
- El animal pertenece al usuario autenticado.

Flujo Principal:

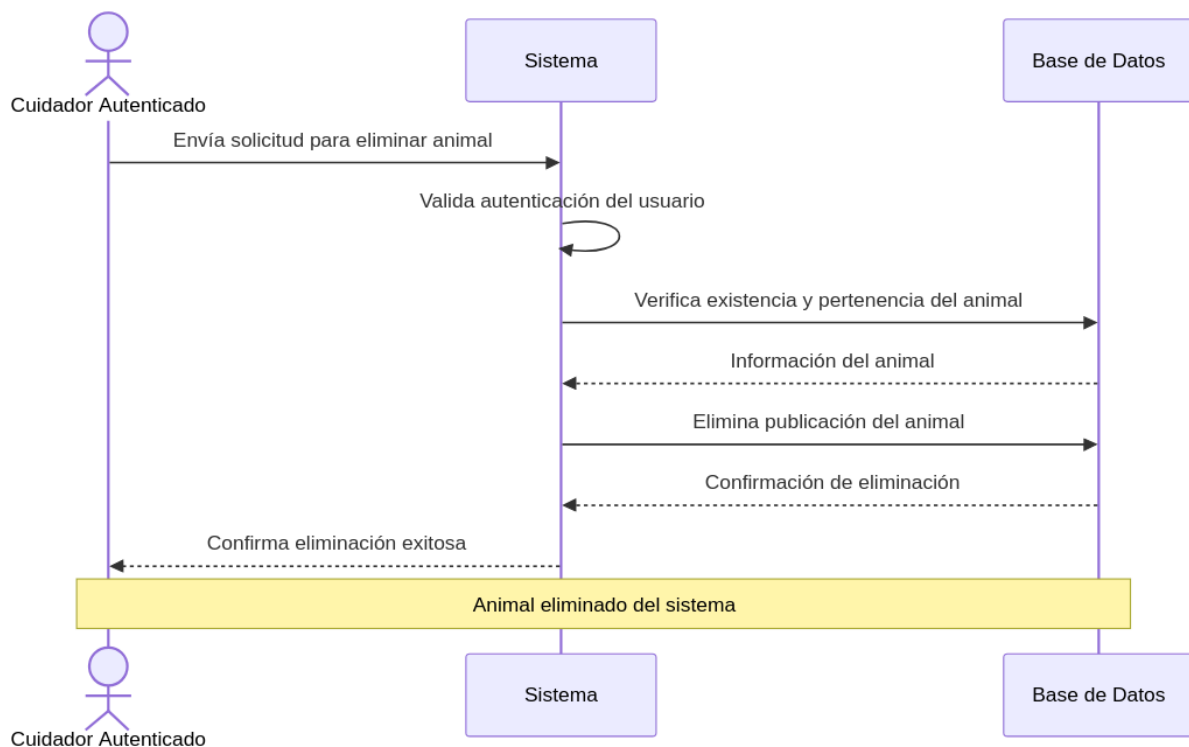
1. El cuidador accede a la vista de administración de publicaciones.
2. El cuidador selecciona un animal registrado y solicita eliminar la publicación.
3. El sistema valida la autenticación y la pertenencia del animal.
4. El sistema elimina la publicación del animal de la base de datos.
5. El sistema confirma la eliminación exitosa.

Flujos Alternativos:

- A1 – Usuario no autenticado: El sistema devuelve un error de autenticación y rechaza la solicitud.
- A2 – Animal no pertenece al usuario: El sistema devuelve un error de autorización y no permite la eliminación.
- A3 – Animal inexistente: El sistema muestra un mensaje indicando que la publicación no existe o ya fue eliminada.

Postcondiciones:

- La publicación del animal deja de existir en el sistema y deja de ser visible en el catálogo.



CU10 – Manifestar interés en un animal (RF12)

Actores:

- Adoptante autenticado
- Sistema de correo electrónico

Descripción: Este caso de uso describe el proceso mediante el cual un adoptante expresa su interés en adoptar un animal, desencadenando una notificación automática al cuidador responsable.

Precondiciones:

- El usuario está autenticado con rol ADOPTANTE.
- El animal existe en el sistema con estatus DISPONIBLE.
- No existe un registro de interés activo del usuario en ese animal al momento de la solicitud.

Flujo Principal:

1. El usuario visualiza la ficha de un animal y hace clic en el botón "Me interesa".
2. El sistema recibe la solicitud con el token de autenticación y el identificador del animal.
3. El sistema valida el token y verifica que el usuario sea ADOPTANTE.
4. El sistema verifica que el animal exista y esté disponible.
5. El sistema verifica que el usuario no haya manifestado interés previamente.
6. El sistema registra el interés en la base de datos.
7. El sistema envía automáticamente un correo al cuidador del animal con los datos de contacto del adoptante, incluyendo al adoptante en copia.
8. El sistema confirma el registro del interés al frontend.
9. El frontend muestra un modal informando al usuario que el cuidador será notificado.

Flujos Alternativos:

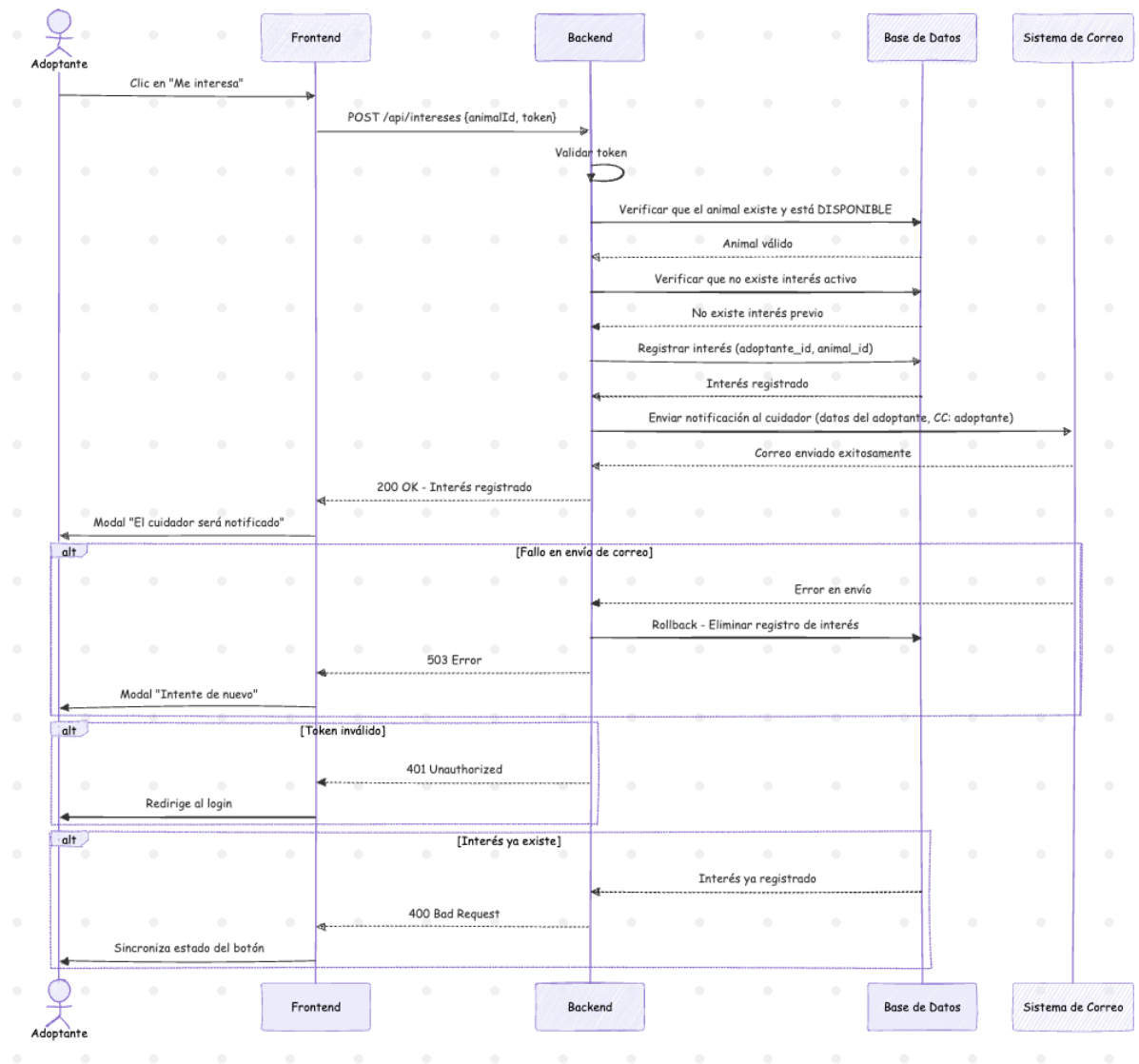
- A1 – Token ausente o inválido: El sistema devuelve un error 401 y rechaza la solicitud.

- A2 – Usuario con rol CUIDADOR: El sistema rechaza la solicitud con un error 400. El botón no se muestra en la interfaz para cuidadores.
- A3 – Animal no encontrado o ya adoptado: El sistema devuelve un error 400. El frontend actualiza el estado del botón indicando que el animal no está disponible.
- A4 – Interés ya registrado: El sistema devuelve un error 400 indicando que el interés ya existe. El frontend sincroniza el estado del botón.
- A5 – Fallo en el envío del correo: El sistema hace rollback del registro de interés y devuelve un error 503. El frontend informa al usuario que debe intentarlo de nuevo.

Postcondiciones:

- El interés queda registrado en la base de datos.
- El cuidador recibe una notificación por correo con los datos de contacto del adoptante.
- El animal aparece en la lista de favoritos del adoptante.

Diagrama:



CU11 – Retirar interés en un animal (RF12)

Actores: Adoptante autenticado, Sistema de correo electrónico

Descripción: Este caso de uso describe el proceso mediante el cual un adoptante retira su manifestación de interés en un animal, notificando automáticamente al cuidador.

Precondiciones:

- El usuario está autenticado con rol ADOPTANTE.
- El usuario tiene un interés registrado en el animal.

Flujo Principal:

1. El usuario hace clic en el botón "En favoritos" (estado activo) en la ficha del animal o en su lista de favoritos.
2. El sistema recibe la solicitud con el token de autenticación y el identificador del animal.
3. El sistema valida el token y verifica que exista un interés registrado.
4. El sistema elimina el registro de interés de la base de datos.
5. El sistema envía automáticamente un correo al cuidador informando que el adoptante retiró su interés, incluyendo al adoptante en copia.
6. El sistema confirma la eliminación al frontend.
7. El frontend muestra un modal informando al usuario que el cuidador ha sido notificado.

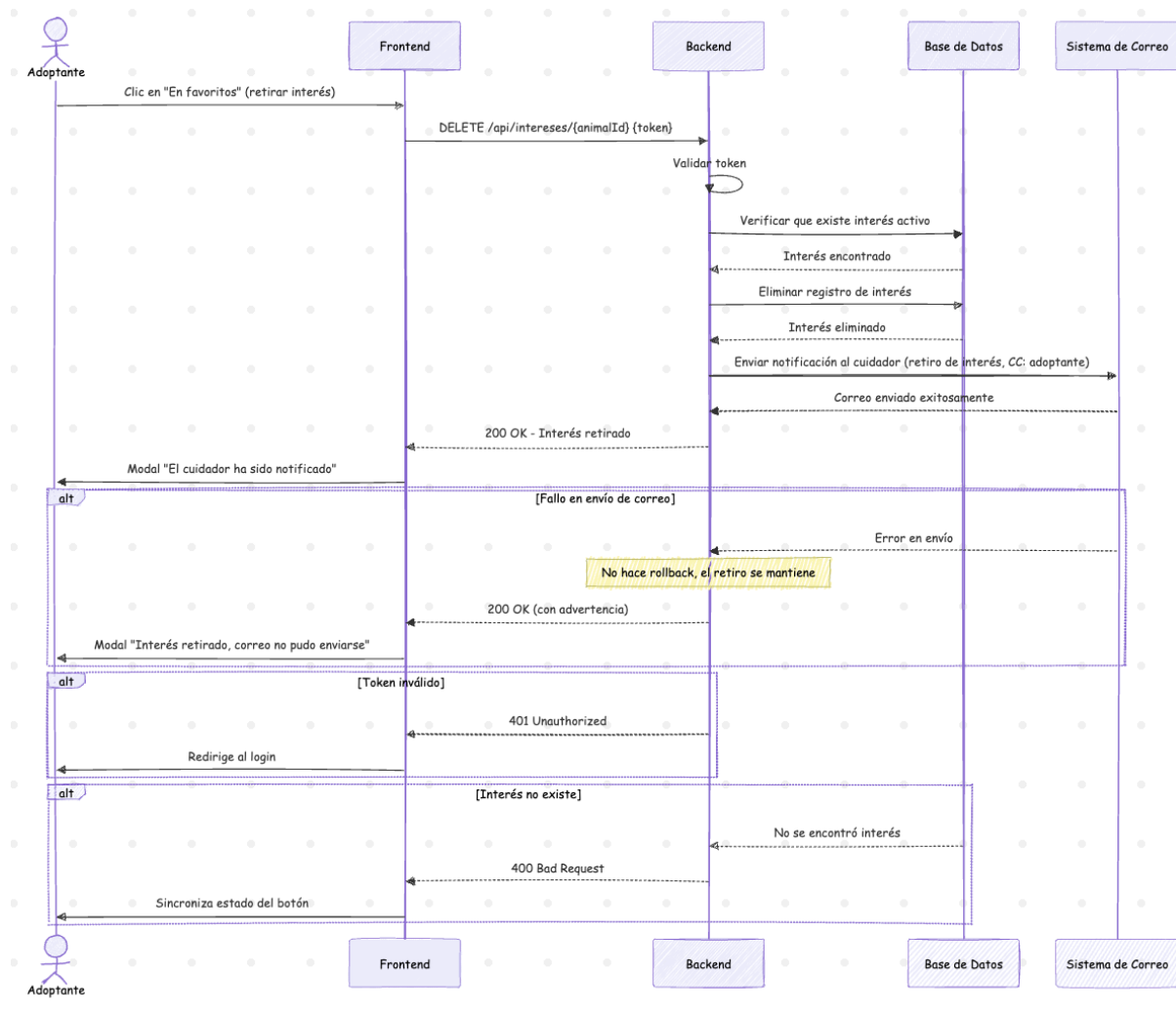
Flujos Alternativos:

- A1 – Token ausente o inválido: El sistema devuelve un error 401 y rechaza la solicitud.
- A2 – Interés no registrado: El sistema devuelve un error 400. El frontend sincroniza el estado del botón.
- A3 – Fallo en el envío del correo: El sistema registra el error, pero no hace rollback. El retiro de interés se completa y se notifica al usuario que el correo no pudo enviarse.

Postcondiciones:

- El interés queda eliminado de la base de datos.
- El cuidador recibe una notificación por correo informando del retiro.
- El animal deja de aparecer en la lista de favoritos del adoptante.

Diagrama:



CU12 – Consultar mis favoritos (RF13)

Actores: Adoptante autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un adoptante consulta la lista de animales en los que ha manifestado interés.

Precondiciones:

- El usuario está autenticado con rol ADOPTANTE.

Flujo Principal:

1. El usuario accede a la vista "Mis favoritos" desde la barra de navegación.
2. El sistema recibe la solicitud con el token de autenticación.
3. El sistema valida el token.

- El sistema recupera todos los animales en los que el usuario ha manifestado interés, incluyendo información básica y foto de portada.
- El frontend muestra las tarjetas de los animales con el botón para retirar el interés.

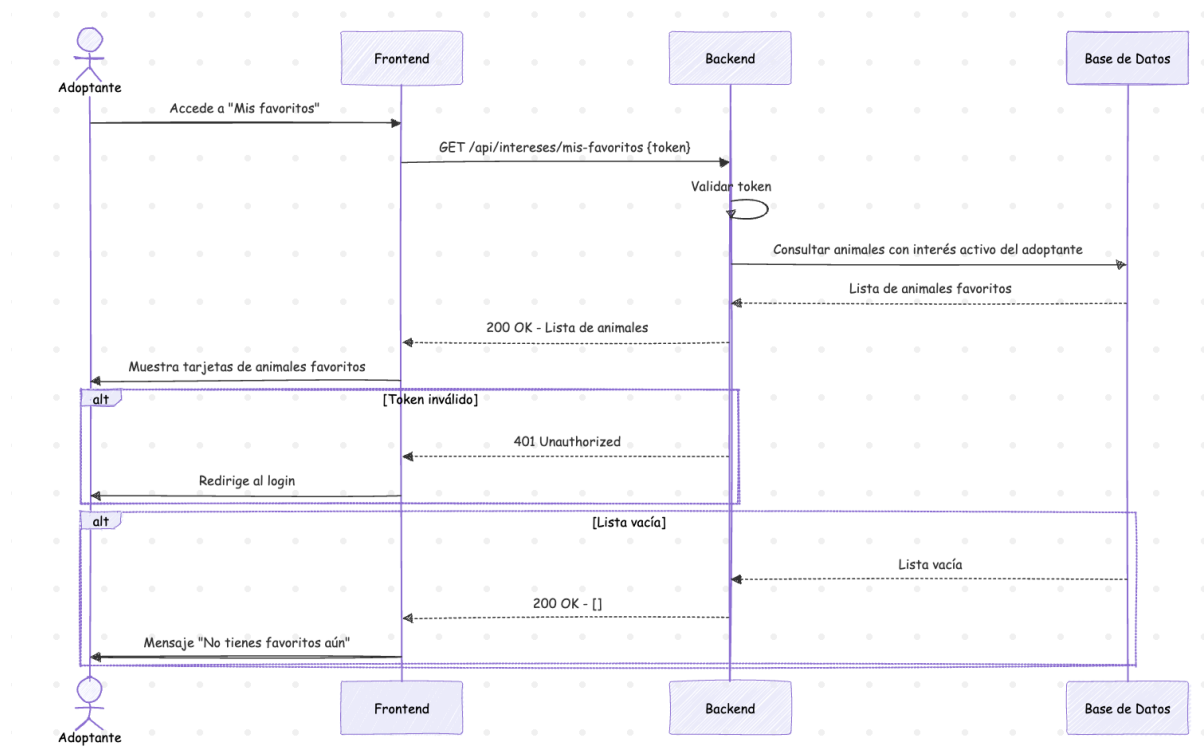
Flujos Alternativos:

- A1 – Token ausente o inválido: El sistema devuelve un error 401. El frontend redirige al login.
- A2 – Lista vacía: El sistema devuelve una lista vacía. El frontend muestra un mensaje indicando que no hay favoritos.

Postcondiciones:

- El usuario visualiza su lista actualizada de animales favoritos.

Diagrama:



CU13 – Marcar animal como adoptado (RF7)

Actores: Cuidador autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un cuidador cambia el estatus de un animal a ADOPTADO, retirándolo del catálogo público.

Precondiciones:

- El usuario está autenticado con rol CUIDADOR.
- El animal pertenece al usuario autenticado.
- El animal tiene estatus DISPONIBLE.

Flujo Principal:

1. El cuidador accede a la vista "Mis mascotas" o al modal de detalle de un animal.
2. El cuidador selecciona la opción "Marcar como adoptado".
3. El sistema recibe la solicitud con el token de autenticación y el identificador del animal.
4. El sistema valida el token y verifica que el animal pertenezca al cuidador.
5. El sistema actualiza el estatus del animal a ADOPTADO en la base de datos.
6. El sistema confirma la actualización al frontend.
7. El frontend actualiza visualmente la tarjeta del animal con indicador de adoptado y lo retira del catálogo público.

Flujos Alternativos:

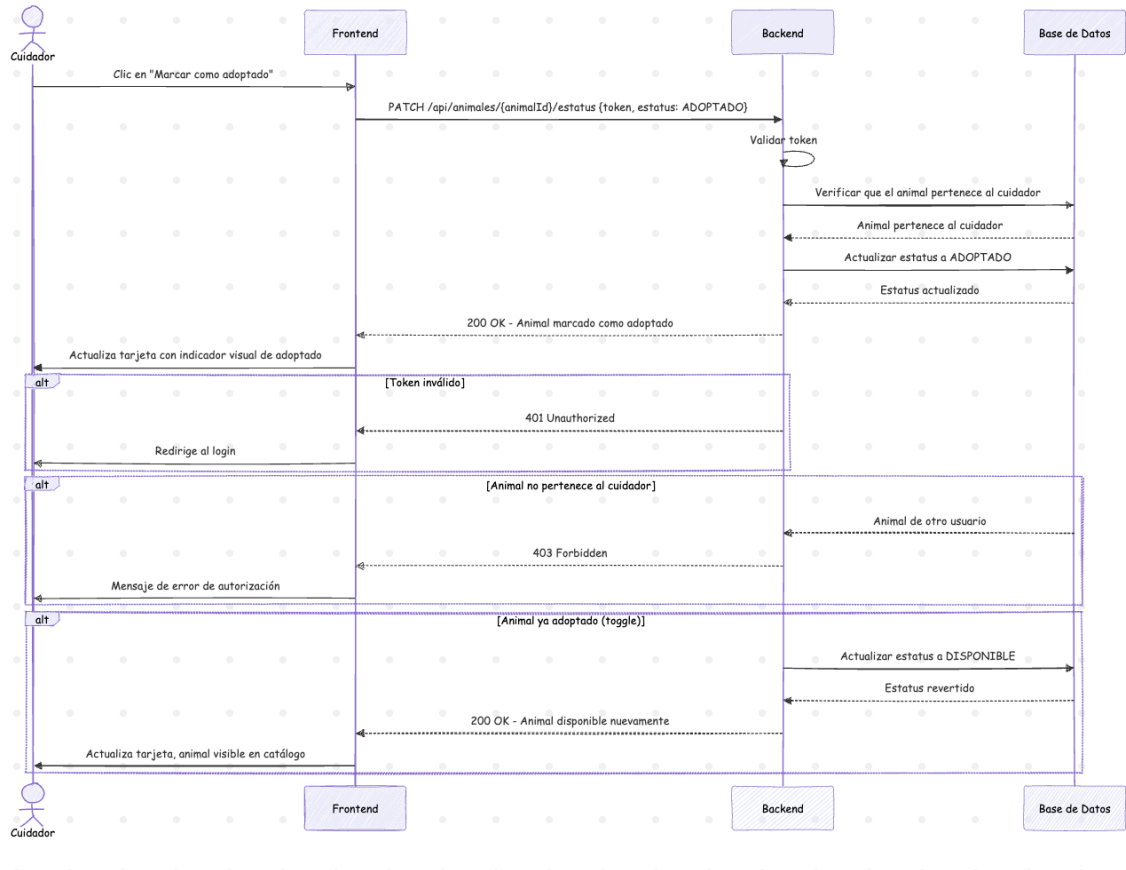
- A1 – Token ausente o inválido: El sistema devuelve un error 401 y rechaza la solicitud.
- A2 – Animal no pertenece al usuario: El sistema devuelve un error 403 y no permite la modificación.
- A3 – Animal ya adoptado: El sistema procesa la solicitud como "Desmarcar adoptado", revirtiendo el estatus a DISPONIBLE y haciéndolo visible nuevamente en el catálogo.

Postcondiciones:

- El animal queda con estatus ADOPTADO y deja de aparecer en el catálogo público.

- El animal permanece visible en "Mis mascotas" del cuidador con indicador visual de adoptado.

Diagrama:



CU14 – Consultar ficha informativa de un animal (RF11)

Actores: Usuario autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un usuario accede a la información completa de un animal registrado en la plataforma.

Precondiciones:

- El usuario está autenticado.
- El animal existe en el sistema.

Flujo Principal:

1. El usuario selecciona un animal desde el catálogo, la vista de exploración o su lista de favoritos.
2. El sistema recibe la solicitud con el identificador del animal.
3. El sistema recupera la información completa del animal: nombre, descripción, fotografías, especie, raza, edad, género, estatus, esterilización, vacunas, padecimientos y fecha de publicación.
4. Si el usuario es el cuidador propietario, el sistema incluye además el número de adoptantes interesados.
5. El frontend muestra la ficha en un modal con galería de fotos navegable, sección de vacunas y padecimientos, y el botón "Me interesa" (para adoptantes) o los controles de gestión (para el cuidador propietario).

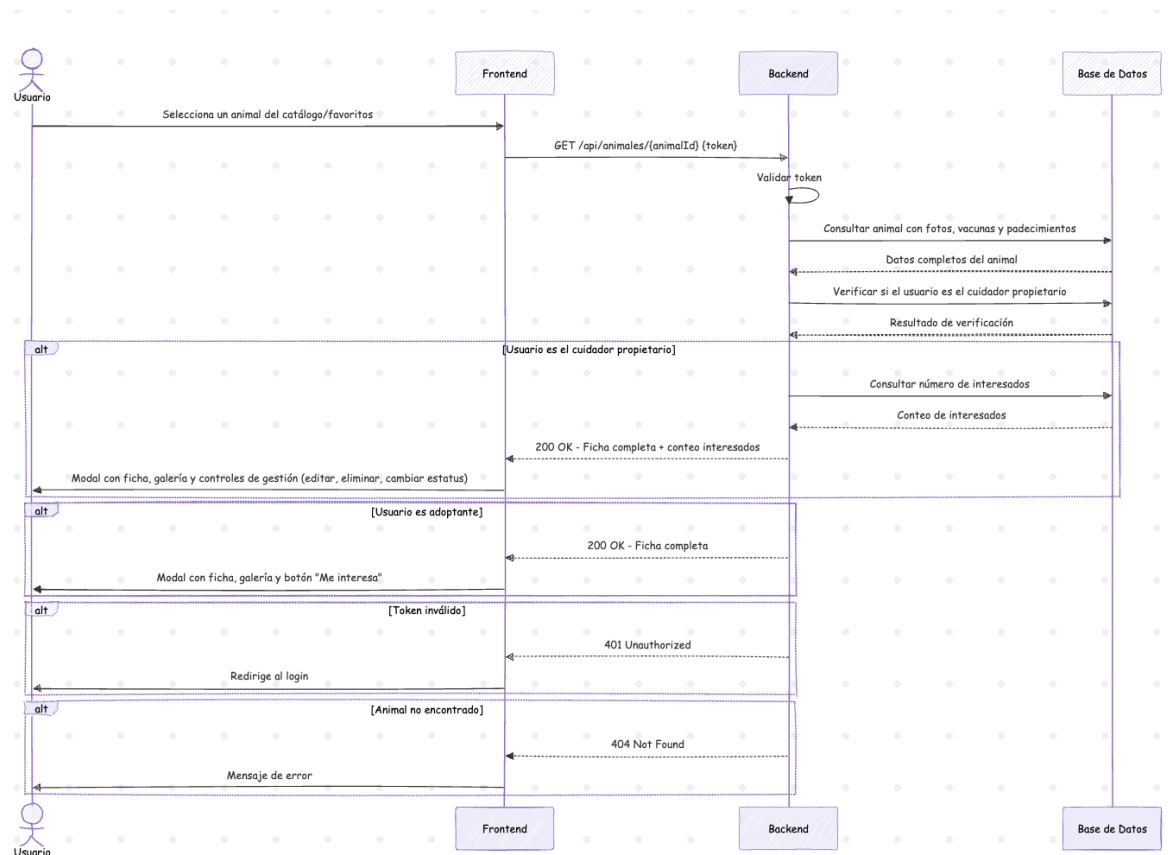
Flujos Alternativos:

- A1 – Token ausente o inválido: El sistema devuelve un error 401. El frontend redirige al login.
- A2 – Animal no encontrado: El sistema devuelve un error 404. El frontend muestra un mensaje de error.
- A3 – ID de animal con formato inválido (no UUID): El sistema devuelve un error 400.

Postcondiciones:

- El usuario visualiza la información completa del animal.
- Si es adoptante, puede manifestar o retirar su interés desde la misma ficha.
- Si es el cuidador propietario, puede editar, eliminar o cambiar el estatus del animal desde la misma ficha

Diagrama:



5. Requerimientos No Funcionales

Tipo		Descripción	Criterios de aceptación	RF
Seguridad	RNF1	Almacenamiento seguro de contraseñas. Las contraseñas deberán almacenarse mediante un algoritmo de cifrado seguro.	Las contraseñas de los usuarios deberán almacenarse de forma irreversible mediante un algoritmo de cifrado unidireccional. En ningún caso se almacenarán en texto plano.	RF1
	RNF2	Ninguna interfaz o funcionalidad del sistema estará disponible para usuarios no autenticados.	El sistema deberá requerir autenticación para acceder a endpoints protegidos. Las solicitudes sin autenticación válida deberán ser rechazadas.	RF2

			El sistema no deberá permitir acceso a información sin autenticación. Los intentos a cualquier URL del sistema sin autenticación previa deberán redirigir al usuario a la página de inicio de sesión. Usuarios no autenticados serán redirigidos a la página de inicio de sesión.	
Accesibilidad y Usabilidad	RNF5	Mostrar errores claros y comprensibles	El sistema debe explicar al usuario que salió mal y qué acción debe tomar para corregirlo. Se debe tener mensajes para errores de formato en registro, errores de inicio de sesión, errores de red, etc.	
		Interfaz atractiva y usable	Los usuarios nuevos deben poder completar una muestra de interés de animal sin asistencia externa en menos de 2 minutos.	
		Interfaz adaptable (responsive)	Debe poder mantener su usabilidad en celular, tablet y escritorio	
		Fácil de usar	El registro de un animal no deberá requerir más de 5 interacciones del usuario (clics o cambios de pantalla).	
Privacidad	RNF6	No mostrar información personal a terceros	La información mostrada a terceros (como código postal) no se pueden usar para identificar de manera única La información personal como curp, nombre completo y contraseña se almacenarán con estricta seguridad y no será disponible para nadie fuera de los administradores del sistema	RF4, RF5

Mantenibilidad			y el propio usuario La comunicación inicial entre usuarios deberá realizarse únicamente mediante el sistema de notificación por correo automático.	
		Privacidad de datos de contacto: Los datos de contacto de los usuarios solo deberán compartirse a través del mecanismo de notificación definido por el sistema. La plataforma no deberá exponer esta información de forma pública ni accesible para terceros no involucrados.	Solo a través del sistema de correo electrónico es que los datos de correo de los usuarios se compartirán.	
	RFN7	El código debe ser modular con una clara separación de capas	Utiliza MVC adaptado a servicios REST	
	RFN8	Pruebas	Incluye pruebas en postman para validar el funcionamiento	

6. Tecnologías Utilizadas

a) Lenguaje de programación en back-end: **Kotlin**

Kotlin fue el lenguaje de programación escogido para el back-end durante los laboratorios de la materia, Kotlin nos brinda compatibilidad con java y podemos usar el ecosistema de bibliotecas de Java, además de tener una sintaxis más clara que otros lenguajes de programación de JVM lo cual nos permite hacer mejores aplicaciones, robustas, seguras y fáciles de mantener; es compatible con frameworks modernos y podemos desarrollar APIs REST.

b) Framework en back-end: **Spring Boot**

Este es el framework que utilizaremos para el back-end, nos ayuda a la creación de aplicaciones REST que sean modulares y podemos tener una arquitectura en capas, además de tener soporte nativo y cuenta con una amplia documentación.

c) Base de datos: **Postgres**

Elegimos esta base de datos dado que es robusta y que la gran parte del equipo ya tenía familiaridad con la misma por lo que eso nos facilita el trabajar todos con ella, además de ser estable, nos brinda confiabilidad, ya que maneja los datos de manera segura.

d) **Hibernate:**

Permite la interacción entre el sistema y la base de datos de manera más eficiente, facilita la conversión de objetos del lenguaje de programación a registros en la base de datos.

e) **Postman:**

Herramienta de prueba para los endpoints, envía solicitudes HTTP y verifica las respuestas del sistema.

f) **Git:**

Sistema de control de versiones la cual nos ayuda a tener un seguimiento de los cambios en el sistema y mantener las versiones estables.

g) Frontend: **NextJS**

Se utilizó Next.js como framework para el desarrollo del frontend debido a su capacidad para construir aplicaciones modernas basadas en **React**, así como su soporte para renderizado eficiente y organización estructurada del proyecto.

h) Framework en front-end: **TailwindCSS**

Se utilizó Tailwind CSS como framework de estilos para agilizar el desarrollo de interfaces consistentes sin necesidad de escribir grandes cantidades de CSS personalizado.

- Además, se utilizó **DaisyUI** como complemento de Tailwind CSS para proporcionar componentes visuales predefinidos

i) Lenguaje de programación en front-end: **TypeScript**

Se utilizó TypeScript como lenguaje de programación en el front-end, el cual extiende JavaScript incorporando tipado estático. Esto permite mejorar la calidad del código y facilitar el mantenimiento del sistema.

El uso de TypeScript contribuye a una mayor robustez en la aplicación, especialmente en la interacción con la API del back-end, al definir estructuras claras para los datos enviados y recibidos.

j) **Mecanismo de comunicación con back-end: Axios**

Se utilizó Axios como biblioteca para la comunicación con el back-end mediante HTTP. Permite centralizar la configuración de las peticiones, manejar respuestas de forma sencilla y aplicar interceptores para la autenticación, evitando la repetición de código en cada llamada.

k) **Cloudinary**: Servicio externo de almacenamiento y optimización de imágenes. Se utiliza para guardar las fotos de perfil de los usuarios y las fotografías de los animales publicados. Proporciona transformaciones automáticas de formato, calidad y dimensiones mediante su SDK para Java (cloudinary-http45) en el backend y @cloudinary/url-gen + @cloudinary/react en el frontend.

l) **Spring Security Crypto (BCrypt)** : Librería del ecosistema Spring utilizada para el cifrado seguro de contraseñas mediante el algoritmo BCrypt. Reemplaza el uso de SHA-256 mencionado en versiones anteriores del documento. BCrypt incluye salt automático, lo que garantiza que contraseñas idénticas produzcan hashes distintos.

m) **Spring Boot Mail / JavaMailSender + Gmail SMTP** : Para mandar correos electrónicos desde la cuenta de colitas felices de Gmail.

n) **Cally**: Web component de código abierto para selección de fechas. Se utiliza en los formularios de publicación y edición de animales. Es compatible con cualquier framework al ser un custom element nativo del browser, y daisyUI lo estiliza automáticamente.

o) **dotenv-kotlin**: Librería para cargar variables de entorno desde un archivo .env en el backend. Permite separar la configuración sensible (credenciales de base de datos, correo, Cloudinary) del código fuente.

7. Arquitectura General

Patrón utilizado: **MVC adaptado a servicios REST**

Este patrón de diseño está implementado mediante una estructura en capas (Modelo, Vista, Controlador) lo cual nos permite una mejor separación de responsabilidades, facilita el mantenimiento y el desarrollo colaborativo. Pero a partir de la Iteración 2, el sistema funciona como un conjunto de dos componentes independientes que solo se comunican entre sí.

Arquitectura en capas:

a) **Controller:**

Es el responsable de recibir las peticiones HTTP del cliente, valida los datos, usa los servicios correspondientes y devuelve la respuesta en formato JSON, este es el intermediario entre el usuario y el interior de la aplicación.

b) **Service:**

Aquí se encuentra el comportamiento de nuestra aplicación, procesa la información mandada por el controller, aplica las reglas de negocio y gestiona procesos.

c) **Repository:**

Es la capa encargada del manejo de la base de datos, facilita la modificación futura del sistema de almacenamiento si fuera necesario, realiza consultas, guarda datos, recupera y actualiza datos.

Componentes del sistema:

a) **Frontend:**

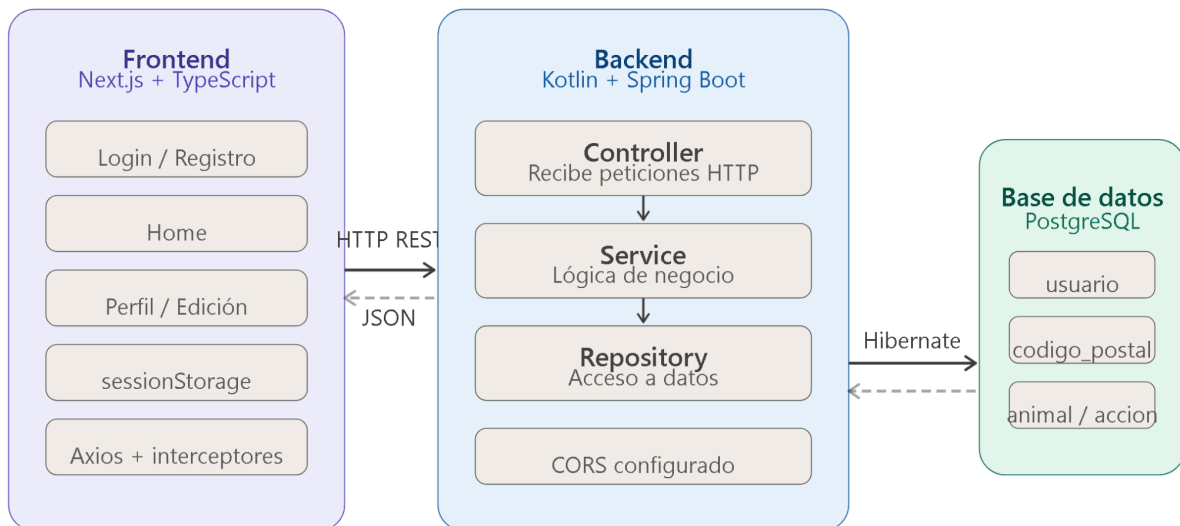
Es la interfaz destinada para el usuario y esta fue desarrollada con Next.js que utiliza la API del backend por medio de peticiones HTTP, el cual puede mostrar las vistas al usuario, gestionar la sesión activa en el navegador y proteger las rutas que requieran autenticación, de esta forma coordina la interacción del usuario con el backend.

b) **Backend:**

API REST se organiza en tres capas, una de ellas es el Controller el cual recibe las peticiones HTTP del frontend, validando los datos y devolviendolos como respuestas en formato JSON, además Service genera tokens, valida las credenciales y aplica las reglas del sistema donde Repository es nuestra capa de acceso a los datos (como la consulta, la actualización, registros), que se guardan en la base de datos mediante Hibernate.

c) **Comunicación:**

El frontend realiza peticiones HTTP al backend utilizando Axios, en donde cada petición que requiera alguna autenticación viene con un token en el encabezado Authorization: Bearer <token>. Además el backend tiene CORS configurado para poder aceptar peticiones del frontend y obtener una respuesta en formato JSON.



8. Flujo de Autenticación

El sistema implementa un flujo de autenticación basado en tokens para gestionar la sesión de los usuarios.

1. El usuario ingresa sus credenciales en la interfaz de login.
2. El front-end envía una solicitud al back-end.
3. El back-end valida las credenciales.
4. Si son correctas, el back-end genera un token de sesión.
5. El front-end almacena el token en sessionStorage.
6. En cada solicitud posterior, el front-end envía el token en el header Authorization.

7. El back-end valida el token antes de permitir acceso a endpoints protegidos.

En caso de que el token no exista o sea inválido:

- El back-end rechaza la solicitud con un error de autenticación.
- El front-end redirige al usuario a la vista de login.

Así, este mecanismo garantiza que solo usuarios autenticados puedan acceder a funcionalidades protegidas del sistema.

9. Modelo Conceptual de Datos

Este sistema se basa en la gestión de varias entidades interrelacionadas que permiten modelar el flujo completo de adopción dentro de la plataforma.

Entidad: Usuario

Esta entidad representa a una persona registrada en el sistema que puede interactuar con la aplicación según su rol asignado.

Atributos principales:

- IDUsuario: UUID. Identificador único de un usuario dentro del sistema. Llave primaria.
- curp: VARCHAR(18). Clave Única de Registro de Población, utilizada para garantizar que cada persona física solo esté asociada a una cuenta.
- username: VARCHAR(50). Nombre con el que se identifica el usuario en su perfil y al interactuar con otros.
- rol: rol_enum con valores ADOPTANTE, CUIDADOR, ADMINISTRADOR. Define el tipo de usuario y los comportamientos disponibles dentro del sistema.
- foto_perfil: VARCHAR(255). URL de Cloudinary que almacena la foto del usuario.
- nombres: VARCHAR(100). Nombre(s) real(es) del usuario registrado.
- apellido_paterno: VARCHAR(100). Apellido paterno del usuario.

- apellido_materno: VARCHAR(100). Apellido materno del usuario.
- email: VARCHAR(100). Dirección de correo electrónico utilizada como identificador de inicio de sesión y medio de notificación.
- codigo_postal: VARCHAR(5). Código postal que permite conocer la ubicación aproximada del usuario sin exponer su dirección exacta.
- password: VARCHAR(255). Contraseña cifrada del usuario.
- token: VARCHAR(255). Token UUID de sesión activa del usuario, generado al iniciar sesión.
- fecha_registro: TIMESTAMP. Fecha y hora en que se creó la cuenta.
- fecha_update: TIMESTAMP. Fecha y hora de la última actualización del perfil.
- fecha_eliminado: TIMESTAMP. Fecha y hora en que se eliminó el perfil (soft delete, reservado para iteraciones futuras).

Entidad: Animal

Esta entidad representa a un animal registrado en la plataforma por un cuidador, disponible para adopción.

Atributos principales:

- IDAnimal: UUID. Identificador único del animal dentro del sistema. Llave primaria.
- nombre: VARCHAR(100). Nombre del animal.
- descripcion: TEXT. Descripción general del animal.
- tipo: tipo_enum con valores PERRO, GATO. Tipo de animal.
- raza: VARCHAR(100). Raza o raza similar del animal.
- fecha_nacimiento: DATE. Fecha de nacimiento del animal, a partir de la cual se calcula la edad en la interfaz.
- genero: genero_enum con valores MACHO, HEMBRA. Género del animal.

- esterilizado: BOOLEAN. Indica si el animal está esterilizado.
- estatus: estatus_enum con valores DISPONIBLE, ADOPTADO. Estado actual de la publicación.
- fecha_publicacion: TIMESTAMP. Fecha y hora en que se publicó el animal.
- fecha_update: TIMESTAMP. Fecha y hora de la última actualización de la ficha.
- id_cuidador: UUID (FK). Referencia al usuario cuidador que publicó el animal. La ubicación del animal se infiere del código postal del cuidador asociado.
- vacunas: Relación N:M con la entidad Vacuna a través de la tabla animal_vacuna. Cada vacuna tiene un nombre único en el catálogo del sistema.
- padecimientos: Relación N:M con la entidad Padecimiento a través de la tabla animal_padecimiento. Cada padecimiento tiene un nombre único en el catálogo del sistema.
- fotos: Relación 1:N con la entidad FotoAnimal. Cada foto almacena una URL de Cloudinary. Un animal puede tener múltiples fotos.

Entidad: Vacuna

Esta entidad representa una vacuna dentro del catálogo del sistema.

Atributos principales:

- vacuna_id: UUID. Identificador único de la vacuna. Llave primaria.
- nombre: VARCHAR(255). Nombre de la vacuna.

Entidad: Padecimiento

Esta entidad representa un padecimiento o condición de salud dentro del catálogo del sistema.

Atributos principales:

- padecimiento_id: UUID. Identificador único del padecimiento. Llave primaria.

- nombre: VARCHAR(255). Nombre del padecimiento.

Entidad: FotoAnimal

Esta entidad almacena las fotografías asociadas a un animal.

Atributos principales:

- foto_id: UUID. Identificador único de la foto. Llave primaria.
- animal_id: UUID (FK). Referencia al animal al que pertenece la foto.
- foto: TEXT. URL de Cloudinary donde se almacena la imagen.
- fecha: TIMESTAMP. Fecha y hora en que se subió la foto.

Entidad: Interés

Esta entidad representa la manifestación de interés de un adoptante en un animal específico.

Atributos principales:

- id_adoptante: UUID (FK). Referencia al usuario adoptante que manifestó interés.
- id_animal: UUID (FK). Referencia al animal en el que se manifestó interés.
- fecha_interes: TIMESTAMP. Fecha y hora en que se registró la manifestación de interés.

Llave primaria compuesta: (id_adoptante, id_animal). No existe un identificador UUID independiente para esta entidad.

Tablas de unión

- animal_vacuna: animal_id UUID (FK), vacuna_id UUID (FK). Relaciona animales con sus vacunas.
- animal_padecimiento: animal_id UUID (FK), padecimiento_id UUID (FK). Relaciona animales con sus padecimientos.

Relaciones

- Un Usuario (cuidador) puede publicar muchos Animales (1:N).
- Un Usuario (adoptante) puede manifestar interés en muchos Animales (N:M a través de Interés con llave compuesta).
- Un Animal puede tener muchas manifestaciones de Interés (1:N).
- Un Animal pertenece a exactamente un Usuario (cuidador) (N:1).
- Un Animal puede tener muchas Fotos (1:N a través de foto_animal).
- Un Animal puede tener muchas Vacunas (N:M a través de animal_vacuna).
- Un Animal puede tener muchos Padecimientos (N:M a través de animal_padecimiento).

Consideraciones de seguridad

Se utiliza BCrypt como algoritmo de cifrado unidireccional con salt automático para el almacenamiento de contraseñas, lo que garantiza que dos contraseñas iguales produzcan hashes distintos y que el valor almacenado no sea recuperable en texto plano a partir del hash almacenado.

10. Limitaciones actuales

- El sistema no cuenta con despliegue en un entorno de producción; su uso se limita a un entorno local.
- El sistema no implementa verificación de correo electrónico al registrarse.
- La foto de perfil se sube después del registro (no durante), ya que preferimos que solo usuarios suban archivos.
- El mapa de animales es un placeholder visual sin funcionalidad real.
- No existe historial dedicado de animales adoptados; los animales marcados como adoptados aparecen en "Mis mascotas" con filtro por estatus, no en una vista separada.

- El almacenamiento de imágenes depende de Cloudinary; sin credenciales configuradas, la subida de fotos no funciona.
- No se implementan filtros ni ordenamientos avanzados en el catálogo de animales.
- No se implementa recuperación de contraseña ni autenticación en dos pasos.
- No se implementan funcionalidades administrativas (moderación de contenido).
- La ficha informativa no incluye datos de APIs externas de razas.